

Day01-熟悉红帽 Linux 与命令行

Day 01 - 熟悉红帽 Linux 与命令行

课程目标

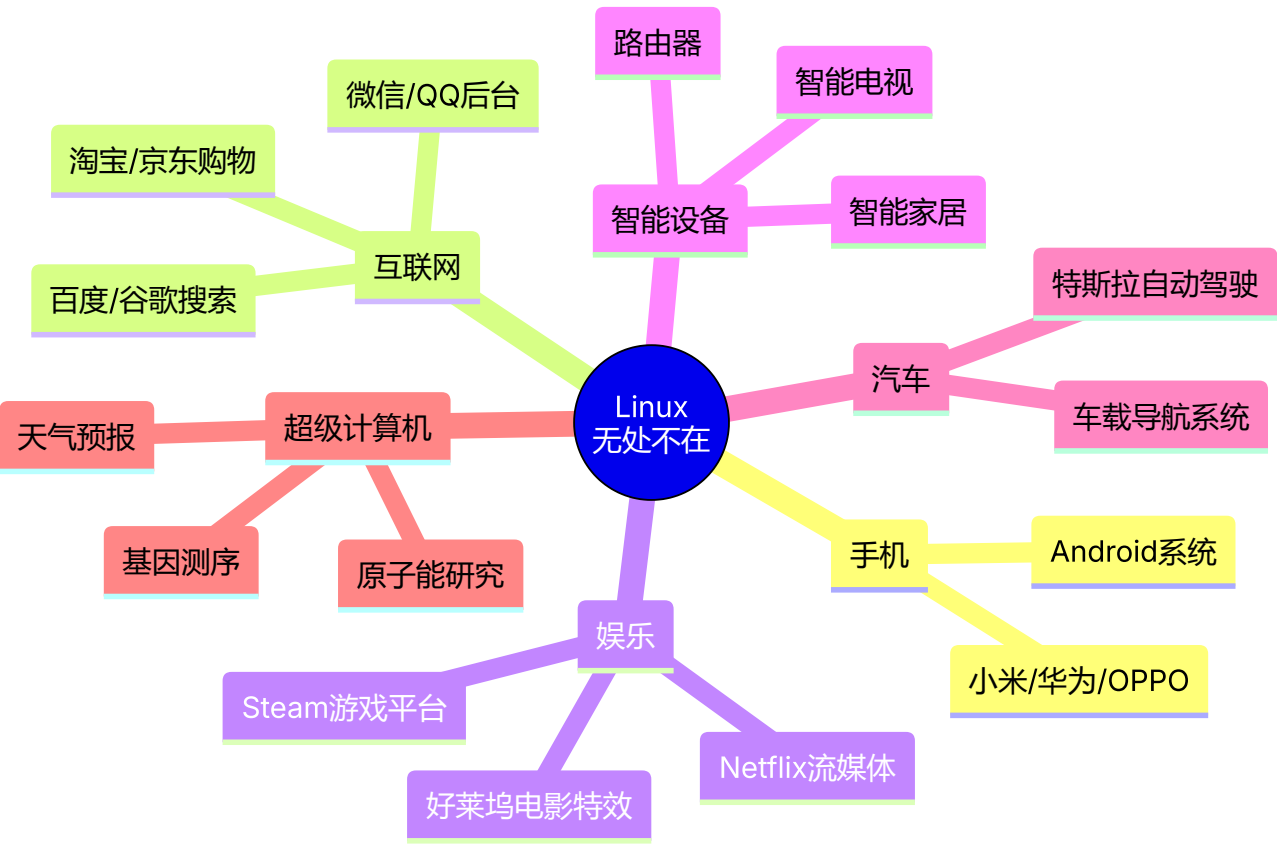
- 了解 RHCSA 认证考试结构和要求
- 完成实验环境准备和配置
- 理解 Linux 的起源、发展史和应用场景
- 掌握 Bash Shell 基础操作
- 学会使用命令行管理文件和目录

上午 - 开班说明

第一部分：开班

1.1 欢迎来到 Linux 世界

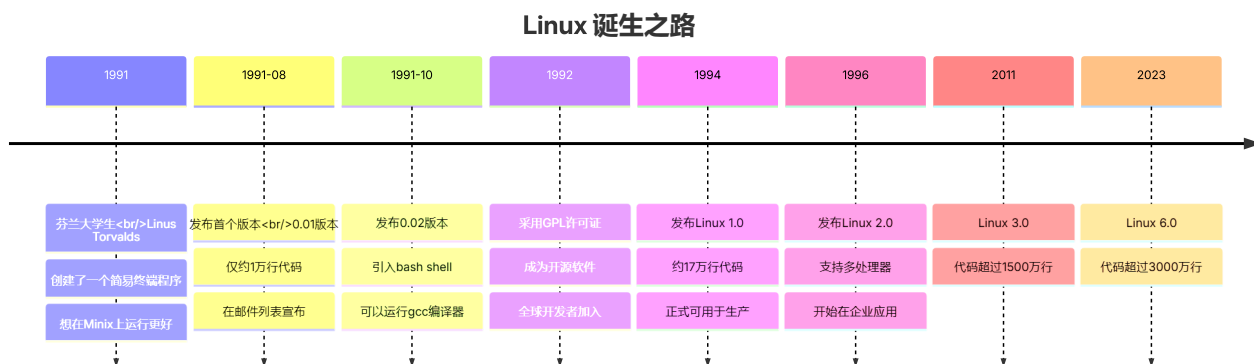
告诉你一个事实：你每天都在使用 Linux！



- 世界排名前500的超级计算机：**100% 运行 Linux**
- 全球云计算服务器：**90%+ 运行 Linux**
- Android 手机的底层内核：**就是 Linux**
- 甚至国际空间站：也使用 **Linux**！

1.2 Linux 的诞生故事

1991年的夏天，芬兰赫尔辛基



那个著名的邮件

1991年8月25日，Linus 在 comp.os.minix 新闻组发布了这条消息：

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Date: 25 Aug 91 20:57:08 GMT
```

Hello everybody out there using minix -

I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready.

"just a hobby, won't be big" —— 这大概是计算机史上最著名的"flag"之一！

1.3 企鹅 Tux 的由来



为什么 Linux 的吉祥物是一只企鹅？

真实故事：

1996年，Linus 去澳大利亚参加会议，在某动物园被一只叫 "Fefe" 的小企鹅咬了一口。他觉得很可爱，就选择企鹅作为 Linux 的吉祥物。

名字 "Tux" 来自：

Tuxedo = 燕尾服

Tux = Tuxedo + Linus Torvalds

为什么不用凶猛的动物？

Linus 说：

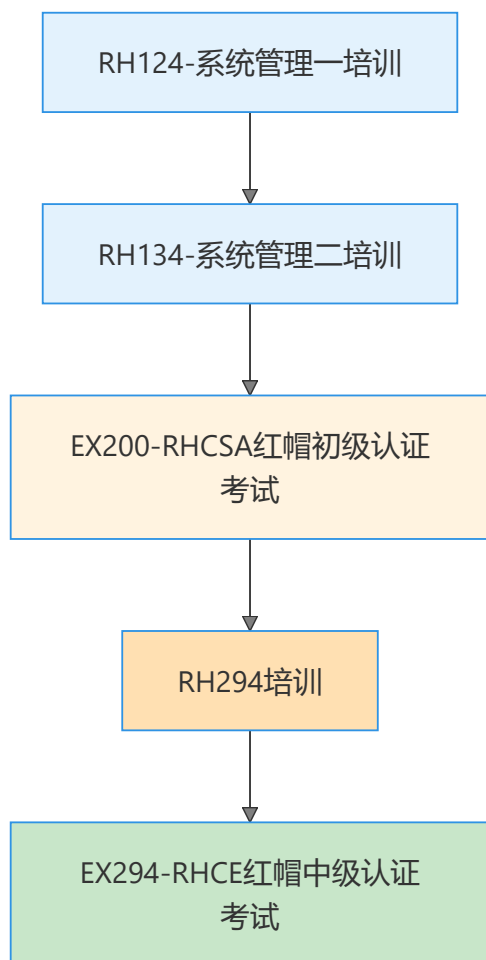
"企鹅看起来很有趣，而且它们不害怕任何东西。

更重要的是，当你看到一只企鹅时，你会忍不住微笑。

这正是 Linux 需要的形象。"

1.4 RHCSA 认证体系

认证路线图



考试信息详解

项目	详细说明
考试代码	EX200
考试时长	3小时)
题目数量	约15-20题
考试形式	远程监考 或 红帽考试中心
及格分数	210分/300分 (70%)
证书有效期	3年
考试费用	¥2800左右 (中国区价格, 具体以官网为准)
出成绩时间	考后3个工作日内

考试通过率

根据红帽官方数据:

- 首次通过率: 约 50-60%
- 重考通过率: 约 70-80%

NIIT学员: 首次 > 80% 重考 > 90%

如何提高通过率?

1. 完成所有课程实验

- 2. 每个知识点都要亲手操作
- 3. 模拟考试环境进行练习
- 4. 掌握常见故障排除
- 5. 时间管理：合理分配每题时间

1.5 学习方法与技巧

学习建议

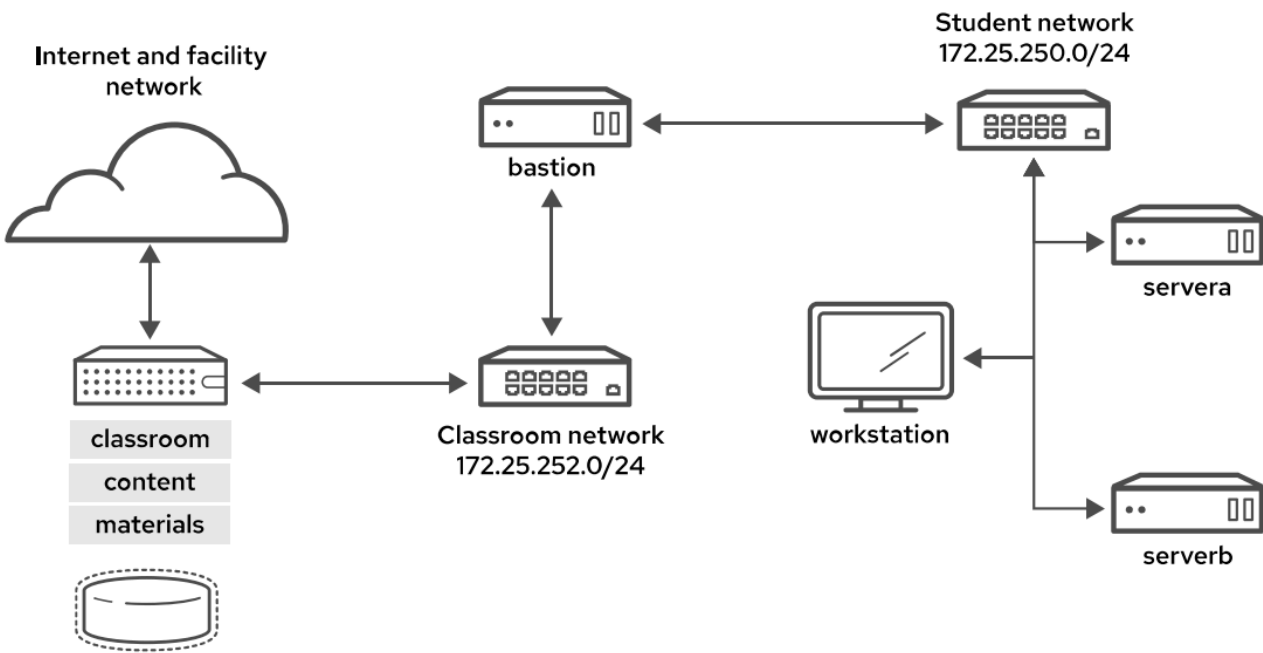
理论与实践结合

- 每个概念都要通过命令验证
- 不要只看不练
- 遇到问题立刻动手解决

第二部分：实验环境深度解析

2.1 环境架构揭秘

完整网络拓扑



主机角色详解表

主机	IP地址	角色	为什么重要
foundation0	172.25.254.250	基础层虚拟机	控制所有课程虚拟机，提供rht-vmctl等管理命令
bastion	172.25.250.254	路由器	连接课堂网络和外部网络，必须一直运行

主机	IP地址	角色	为什么重要
workstation	172.25.250.9	你的工作机	完成日常练习的主要环境
servera	172.25.250.10	练习服务器A	模拟生产服务器，练习服务器管理
serverb	172.25.250.11	练习服务器B	用于设计到存储/分区/挂载相关练习
utility	172.25.250.220	工具服务器	提供额外服务（如容器仓库）
classroom	172.25.254.254	课程服务器	提供学习资源和其他服务，需要一直运行

域名解析规则

SHELL

```
# 完整域名格式
hostname.lab.example.com

# 简写（配置hosts后）
workstation
servera
serverb

# 它们会自动解析为
workstation.lab.example.com → 172.25.250.9
servera.lab.example.com     → 172.25.250.10
serverb.lab.example.com     → 172.25.250.11
```

2.2 环境管理命令全解

1. 查看当前课程环境

SHELL

```
# 显示当前课程环境
cat /etc/rht
```

命令详解：

- `cat` - 查看文件内容的命令
- `/etc/rht` - 存储课程信息的配置文件路径

预期输出：

```
RH124
```

输出解读：

- `RH124` - 表示当前系统配置为 RH124（系统管理一）环境
- 如果显示 `RH134` - 则是系统管理二环境
- 如果显示 `RH294` - 则是 RHCE 加速环境

2. 查看所有可用环境

```
# 列出所有可用课程
ls /content/manifest/RH*
```

SHELL

命令详解:

- `ls` - 列出目录内容的命令
- `/content/manifest/` - 存储课程清单的目录
- `RH*` - 通配符, 匹配所有以 RH 开头的文件或目录

预期输出:

```
/content/manifest/RH124
/content/manifest/RH134
/content/manifest/RH294
```

输出解读:

- `RH124` - 系统管理一课程
- `RH134` - 系统管理二课程
- `RH294` - RHCE 课程

3. 切换课程环境

如何需要切换课程, 首先需要清除原来的课程环境

```
# 清除当前环境 (谨慎使用!)
rht-clearcourse 0
```

SHELL

命令详解:

- `rht-clearcourse` - 红帽培训环境管理命令
- `0` - 参数, 表示清除所有课程配置

⚠ 警告: 此命令会清除当前课程的所有配置!

什么时候需要清除?

- 要切换到完全不同的课程时
- 环境出现严重问题需要重置时

然后设置期望的课程环境:

```
# 设置课程环境为 RH124
rht-setcourse rh124
```

SHELL

命令详解:

- `rht-setcourse` - 设置课程环境命令
- `rh124` - 要设置的课程名称 (小写)

预期输出:

```
Setting course to RH124...
Course set successfully.
```

输出解读:

- 看到 `successfully` 表示设置成功
- 系统会自动配置 RH124 相关的虚拟机和实验环境

验证步骤:

```
# 再次查看当前环境确认
cat /etc/rht
# 应该显示: RH124
```

SHELL

4. 启动虚拟机

⚠ 重要: 虚拟机启动有正确顺序!

```
# 步骤1: 保证 classroom 虚拟机正常运行 (课程服务器一般会运行)
rht-vmctl start classroom

# 步骤2: 再启动所有其他课程虚拟机
rht-vmctl start all
```

SHELL

为什么需要先启动 `classroom`?

启动顺序的重要性:

`classroom` (课程服务器)

- └─ 提供学习资源
- └─ 提供实验评分服务
- └─ 管理课程网络
- └─ 必须一直运行

其他虚拟机 (`workstation`、`servera`等)

- └─ 依赖 `classroom` 提供的服务
- └─ `classroom` 启动后再启动它们

正确顺序:

`classroom` → `all` (`bastion` + `workstation` + `servera` + `serverb`)

命令详解:

- `rht-vmctl` - 红帽虚拟机控制命令 (Red Hat VM ConTroL)

- `start classroom` - 先启动课程服务器
- `start all` - 再启动所有其他虚拟机

预期输出：

```
Starting classroom...
Starting bastion...
Starting workstation...
Starting servera...
Starting serverb...
```

输出解读：

- 先启动 classroom（课程服务器）
- 系统会逐个启动所有课程虚拟机
- 这个过程可能需要10分钟左右

```
# 仅启动 servera 虚拟机
rht-vmctl start servera
```

SHELL

命令详解：

- `start` - 启动动作
- `servera` - 指定要启动的虚拟机名称

预期输出：

```
Starting servera...
```

使用场景：

- 只需要在一台机器上练习时
- 节省系统资源（不启动不需要的虚拟机）

5. 查看虚拟机状态

```
# 查看所有虚拟机状态
rht-vmctl status all
```

SHELL

命令详解：

- `status` - 查看状态动作
- `all` - 所有虚拟机

预期输出：

```
workstation      running (since 2024-01-15 09:00)
servera          running (since 2024-01-15 09:01)
serverb          stopped
bastion          running (since 2024-01-15 08:58)
```

输出解读：

状态	含义	能否SSH连接
running	正在运行	✅ 不一定 (需要等待完全启动就绪后)
stopped	已停止	❌ 不可以
undefined	未定义/不存在	❌ 不可以

```
# 查看单个虚拟机状态
rht-vmctl status servera
```

SHELL

预期输出：

```
servera          running (since 2024-01-15 09:01)
```

验证步骤：

```
# 如果显示 running, 尝试SSH连接测试
ssh root@servera
# 密码: redhat

# 登录后执行 hostname 确认
hostname
# 应该显示: servera

# 退出连接
exit
```

SHELL

6. 重启虚拟机

```
# 重启 servera 虚拟机
rht-vmctl restart servera
```

SHELL

命令详解：

- `restart` - 重启动作
- `servera` - 要重启的虚拟机

预期输出：

```
Stopping servera...
Starting servera...
```

说明： 相当于按下虚拟机的"重启按钮"

使用场景：

- 虚拟机出现异常但不想完全重置
- 测试服务开机自启动
- 应用某些配置后需要重启

验证步骤：

```
# 等待30秒后检查状态
rht-vmctl status servera
# 应该显示: running
```

SHELL

7. 重置虚拟机（危险操作！）

```
# 重置虚拟机到初始状态
rht-vmctl reset servera
```

SHELL

命令详解：

- **reset** - 重置动作，将虚拟机恢复到初始状态
- **servera** - 要重置的虚拟机名称

⚠ 严重警告：

- ✓ 会将虚拟机恢复到初始状态
- × 所有配置丢失
- × 所有数据丢失
- × 所有安装的软件被删除

使用场景：

- 虚拟机严重损坏无法修复
- 实验环境混乱需要重新开始
- 明确知道后果并愿意承担

8. 实验练习系统

8.1 开始实验

```
# 开始一个名为 "managing-files" 的实验
lab start managing-files
```

命令详解:

- **lab** - 红帽实验管理系统命令
- **start** - 开始实验动作
- **managing-files** - 实验名称（管理文件），不同培训环境实验内容不同, tab会有补全提示, 或者参考教材来完成。

输出解读:

- 系统会自动创建练习所需的文件和目录
- 设置初始状态供你练习
- 准备就绪后提示设置完成

验证步骤:

```
# 查看实验说明（在教材对应章节后面）

# 准备实验环境，通过lab start 按tab键可以补全，这里以假设存在实验 managing-files
lab start managing-files
```

8.2 评分实验

```
# 评估 "managing-files" 实验的完成情况
lab grade managing-files
```

命令详解:

- **grade** - 评分动作
- **managing-files** - 要评分的实验名称

预期输出:

```
Grading exercise: managing-files

Task 1: Create directory..... PASS
Task 2: Set permissions..... PASS
Task 3: Copy files..... FAIL
Task 4: Set ownership..... PASS

Overall score: 3/4 (75%)
```

输出解读:

- **PASS** - 任务完成 
- **FAIL** - 任务未完成 
- 会详细列出每个任务的得分情况

8.3 完成实验

```
# 完成 "managing-files" 实验, 清理环境
lab finish managing-files
```

SHELL

命令详解:

- **finish** - 完成动作
- 会清理实验创建的文件和环境
- 为下一个实验做好准备

输出解读:

- 清理实验相关的所有文件
- 恢复系统到实验前状态
- 准备好开始下一个实验

重要提醒:

- ⚠ 执行 finish 后, 实验环境会被清理
- ⚠ 确保完成评分后再执行 finish
- ⚠ 如果需要继续练习, 不要执行 finish

最佳实践流程:

```
# 推荐的实验完整流程
lab start exercise-name      # 1. 开始实验
# ... 进行练习 ...
lab grade exercise-name      # 2. 评分检查
# ... 如有错, 修正后再次评分 ...
lab grade exercise-name      # 3. 再次评分直到满分
lab finish exercise-name     # 4. 完成并清理
```

SHELL

2.2.5 虚拟机生命周期管理

⚠ 重要提示: reset 命令的风险

rht-vmctl reset servera 的后果:

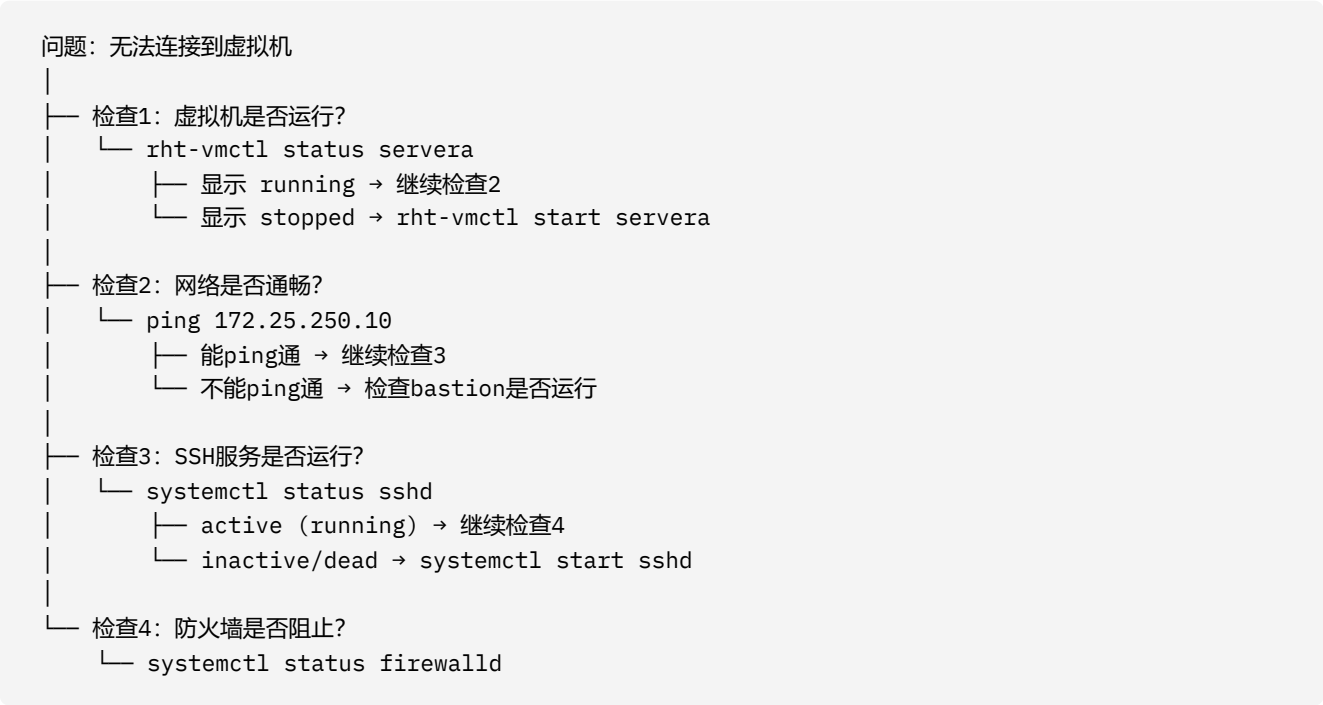
- ✓ 会将虚拟机恢复到初始状态
- ✗ 所有配置丢失
- ✗ 所有数据丢失
- ✗ 所有安装的软件被删除

仅在以下情况使用:

- 虚拟机严重损坏无法修复
- 需要重新开始某个实验
- 明确知道后果并愿意承担

2.2.6 故障排查指南

故障排查思维导图



常见问题速查表

现象	可能原因	快速诊断	解决方案
SSH拒绝连接	虚拟机未启动	rht-vmctl status	rht-vmctl start后等几分钟
SSH连接超时	网络不通	ping IP地址	检查bastion，重启网络
权限被拒绝	密码错误	重新输入密码	确认用户名和密码
命令不存在	用户错误	whoami	切换到正确用户
hostname不正确	登错机器	hostname	exit 重新登录

2.3 配置主机文件

培训环境应该已经完成了主机文件配置了， 如果没有， 可以按照如下方式配置：

为什么需要配置 /etc/hosts？

问题场景：

没有hosts配置时：

```
ping servera
# 错误: ping: servera: Name or service not known
# 系统不知道 servera 是谁
```

配置hosts后：

```
ping servera
# 成功: 64 bytes from servera (172.25.250.10)
# 系统知道 servera = 172.25.250.10
```

配置步骤详解

步骤1：确认当前是root用户

```
# 确认当前用户身份
whoami
```

SHELL

输出说明：

- 应该显示：`root`
- 提示符应该是 `#` 而不是 `$`

步骤2：添加主机记录

```
# 添加缺失的主机记录到 /etc/hosts 文件
172.25.250.9    workstation.lab.example.com workstation
172.25.250.10   servera.lab.example.com servera
172.25.250.11   serverb.lab.example.com serverb
172.25.254.254  bastion.lab.example.com bastion
172.25.250.220  utility.lab.example.com utility
172.25.250.220  registry.lab.example.com registry
```

SHELL

说明：

- `>>` 表示追加到文件末尾
- `<< 'EOF'` 是 heredoc 语法，用于输入多行内容
- 单引号 `'EOF'` 表示不解析变量

步骤3：测试域名解析

```
# 测试 workstation 解析
ping -c 2 workstation
```

SHELL

输出示例：

```
PING workstation.lab.example.com (172.25.250.9) 56(84) bytes of data.
64 bytes from workstation.lab.example.com (172.25.250.9): icmp_seq=1 ttl=64 time=0.123 ms
```

```
# 测试 servera 解析
ping -c 2 servera
```

SHELL

```
# 测试 serverb 解析
ping -c 2 serverb
```

hosts 文件格式说明

格式: IP地址 完整域名 别名1 别名2 ...

示例:

```
172.25.250.10  servera.lab.example.com  servera
    ↓           ↓           ↓
  IP地址      完整域名      简短别名
```

注意事项:

1. IP地址和域名之间用空格或Tab分隔
2. # 开头的行为注释
3. 一行可以配置多个别名
4. 文件从上往下匹配, 先匹配到的生效

2.4 SSH 连接配置

SSH 是什么?

SSH = Secure Shell (安全外壳协议)

作用: 加密的远程登录协议

发明人: Tatu Ylönen (芬兰, 1995年)

为什么需要: 传统的telnet明文传输, 极不安全

SSH 的工作原理:

你的电脑		服务器
↓		↓
[发起连接]	——加密——>	[验证身份]
[发送密码]	——加密——>	[验证通过]
[执行命令]	——加密——>	[返回结果]

使用SSH 连接

方式1: 使用 IP 地址登录

```
# 基本密码登录
ssh student@172.25.250.9
```

说明:

- 会提示输入密码: **student**
- **172.25.250.9** 是 workstation 的 IP 地址

- 没有做额外配置的情况下，默认访问与当前登录用户相同的用户

方式2：使用主机名登录

```
# 使用主机名（需先配置 /etc/hosts）  
ssh student@workstation
```

SHELL

优点：更方便，不用记 IP 地址

方式3：指定端口登录

```
# 指定端口登录  
ssh -p 2222 student@workstation
```

SHELL

说明：

- 默认端口是 22
 - `-p` 指定其他端口
-

方式4：执行远程命令

```
# 不登录，直接在远程执行命令  
ssh student@servera 'hostname'
```

SHELL

输出： `servera`

方式5：执行多行命令

```
# 执行远程脚本  
ssh student@servera << 'REMOTE_CMD'  
hostname  
whoami  
date  
REMOTE_CMD
```

SHELL

说明：执行多行命令

（选学）SSH 配置文件优化

培训环境中已经完成了类似如下配置，用于简化特定场景下的 SSH 连接。

```
# SSH 配置文件
vim ~/.ssh/config
```

SHELL

配置内容示例：

```
Host workstation
  HostName 172.25.250.9
  User student
  Port 22

Host servera
  HostName 172.25.250.10
  User student

Host serverb
  HostName 172.25.250.11
  User student
```

SHELL

配置后使用：

```
# 直接使用主机名，无需输入用户名和 IP
ssh workstation
```

SHELL

下午 - Linux 基础与命令行

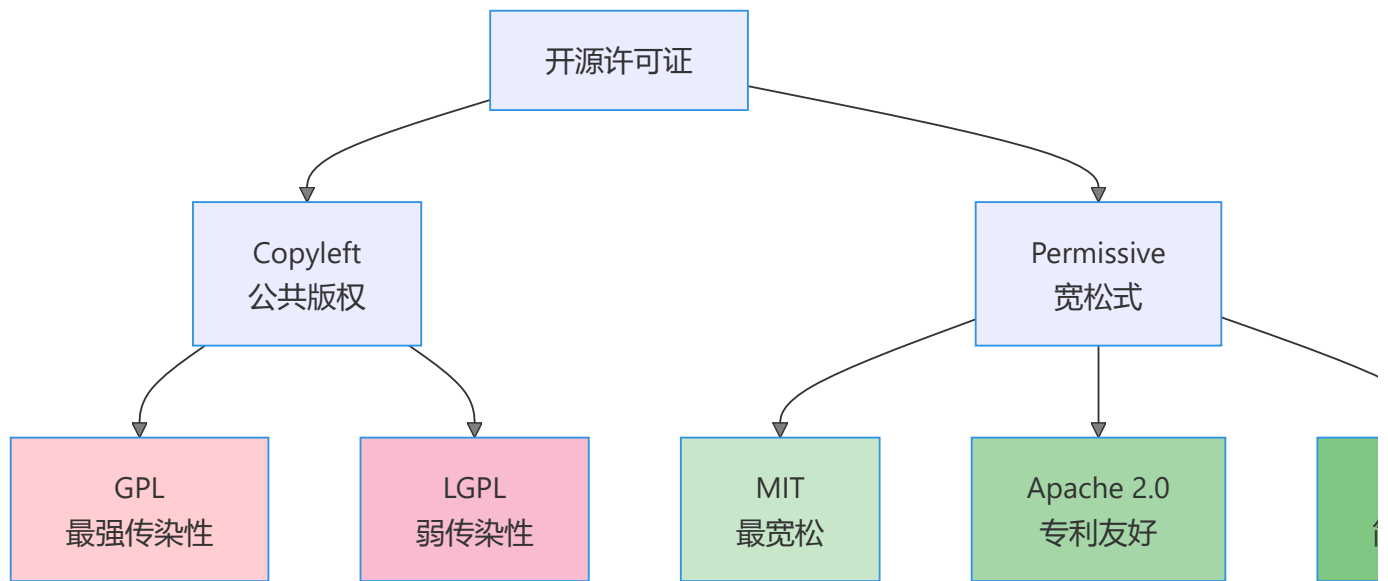
第三部分：Linux 世界

3.1 开源软件的神奇世界

什么是开源？

```
开源软件 (Open Source Software)
├─ 定义：源代码公开可用的软件
├─ 特点：
│   ├── 自由使用
│   ├── 自由研究
│   ├── 自由修改
│   └── 自由分发
└─ 核心："开源不等于免费"
```

开源许可证详解



许可证选择决策树

你的项目需要开源?

- ├─ 希望: 保持代码永远开源
 - ├─ 选择: GPL
 - ├─ 优点: 确保改进也开源
 - └─ 缺点: 商业应用受限
- ├─ 希望: 允许商业使用
 - ├─ 最宽松: MIT
 - ├─ 优点: 极简, 几乎无限制
 - └─ 缺点: 无专利保护
 - ├─ 专利保护: Apache 2.0
 - ├─ 优点: 明确专利授权
 - └─ 缺点: 条款较多
 - ├─ 中等: BSD
 - ├─ 优点: 简单宽松
 - └─ 缺点: 不如MIT流行
- └─ 特殊需求
 - ├─ 库文件: LGPL
 - ├─ 允许闭源链接
 - └─ 文档/教程: CC BY

著名开源项目

你可能每天都在用的开源软件：

浏览器：

- └ Chrome → Chromium (开源)
- └ Firefox → Mozilla (开源)
- └ Edge → Chromium (开源)

操作系统：

- └ Android → Linux内核 (开源)
- └ macOS → Darwin (部分开源)
- └ Windows → WSL (Linux子系统)

编程语言：

- └ Python → 开源
- └ Java → OpenJDK (开源)
- └ Go → 开源
- └ Rust → 开源

服务器软件：

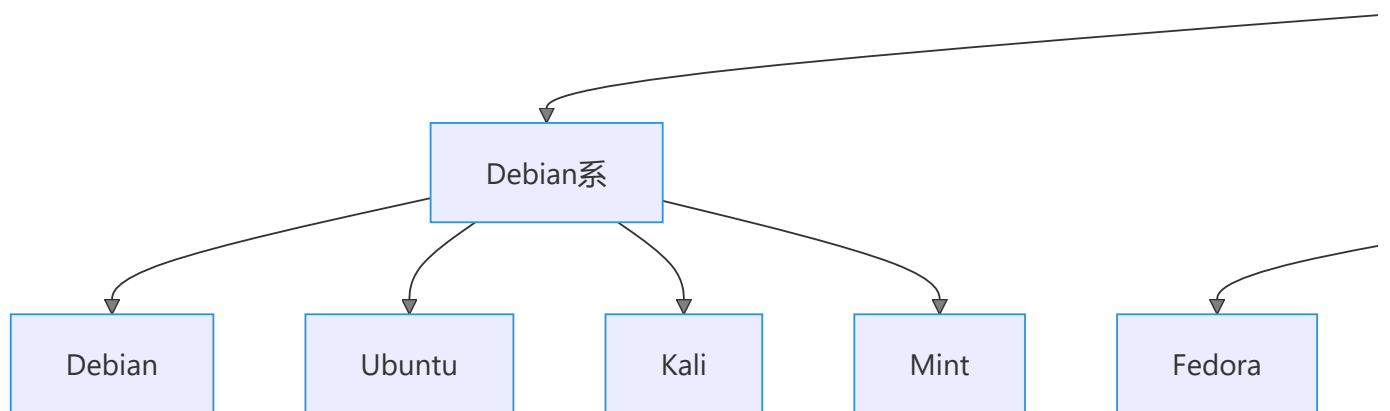
- └ Nginx → 开源
- └ Apache → 开源
- └ MySQL → 开源
- └ PostgreSQL → 开源

容器技术：

- └ Docker → 开源
- └ Kubernetes → 开源

3.2 Linux 发行版家族

发行版族谱

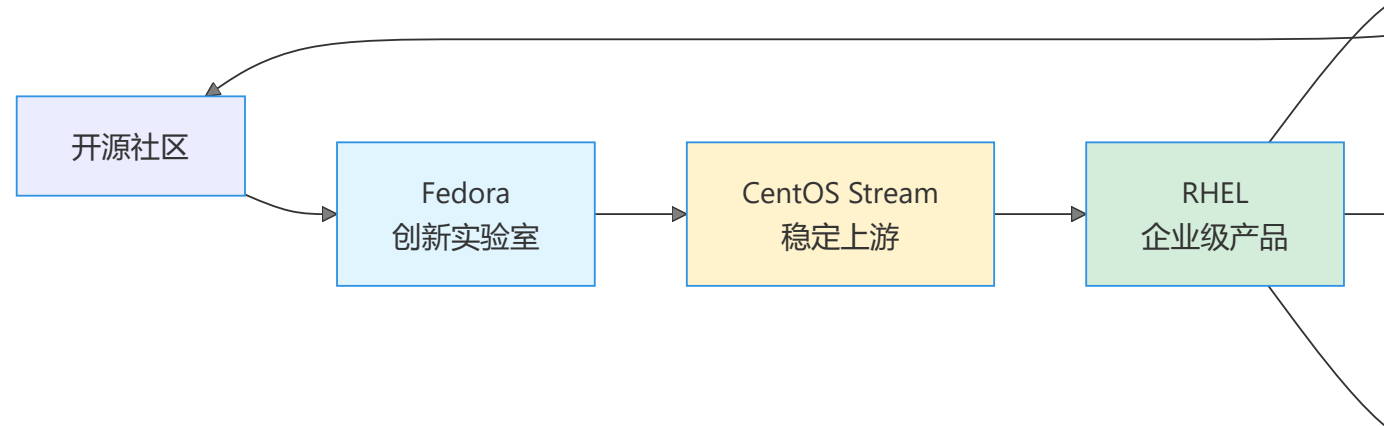


主流发行版对比

发行版	包管理	特点
Ubuntu	apt	易用，社区大
Debian	apt	稳定，保守
Fedora	dnf	最新技术

发行版	包管理	特点
RHEL	dnf	企业级，付费订阅
CentOS Stream	dnf	RHEL上游
Arch	pacman	滚动更新

红帽生态系统详解



红帽系Linux产品生命周期

Fedora 发布周期

- 版本发布：每6个月
- 支持周期：约13个月
- 更新内容：最新软件
- 风险等级：较高（可能不稳定）

CentOS Stream

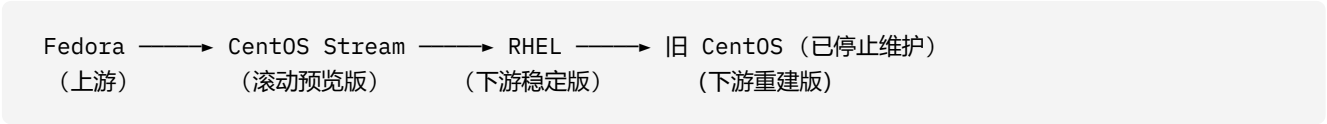
- 版本类型：滚动更新
- 稳定性：测试后发布
- 作用：RHEL预览
- 适合：开发测试

RHEL

- 主版本：每2-3年
- 次版本：约6个月
- 支持周期：10年
 - 全支持：5年
 - 维护支持：5年
 - 延长支持：额外5年
- 适合：生产环境

3.3 CentOS 与 CentOS Stream 的区别

核心关系



对应 RHEL 版本关系

当前版本 (RHEL 10):



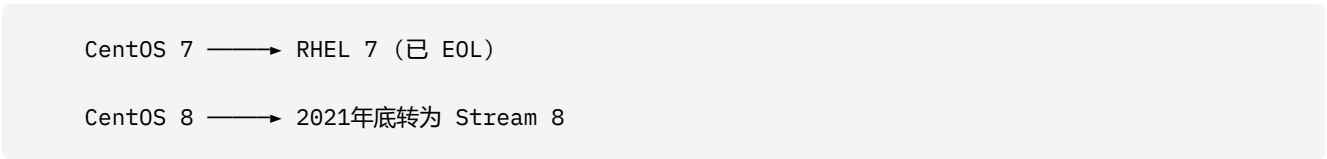
RHEL 9 系列:



RHEL 8 系列:



历史版本 (已停止支持):



主要区别

特性	CentOS (传统版)	CentOS Stream
定位	RHEL 的下游重建版	RHEL 的上游滚动发布版
更新模式	滞后 RHEL 发布	领先 RHEL 发布
生命周期	固定版本 (如 7/8)	持续滚动更新
支持状态	CentOS7在2024年6月30日停止 CentOS8在2022年1月1日停止	✅ 当前唯一官方版本
适用场景	追求极致稳定	跟上新技术、提前测试

CentOS Stream 的特点:

- 更新更快, 包含 RHEL 即将发布的功能
- 适合开发测试环境、学习新技术
- 不建议 用于关键生产环境 (稳定性未完全验证)

替代方案:

- Rocky Linux / AlmaLinux** → 传统 CentOS 的直接替代, 追求与 RHEL 1:1 兼容
- CentOS Stream** → 官方推荐, 适合非核心业务
- 国产替代: openEuler(华为欧拉)/Anolis(阿里龙蜥) - 自主可控, 符合信创要求

简单总结

版本	状态	特点
旧 CentOS	已停止	稳定但滞后于 RHEL
CentOS Stream	官方版本	滚动更新, 领先 RHEL
Rocky/AlmaLinux	社区替代	追求与 RHEL 1:1 兼容

第四部分：命令行基础

4.1 图形界面 vs 命令行

为什么管理员喜欢命令行?

图形界面（GUI）的特点：

- └─ 直观易懂
- └─ 适合新手
- └─ 鼠标操作
- └─ 缺点：
 - └─ 资源占用高
 - └─ 操作效率低
 - └─ 难以自动化
 - └─ 远程管理不便

命令行（CLI）的特点：

- └─ 需要记忆命令
- └─ 适合专业人员
- └─ 键盘操作
- └─ 优点：
 - └─ 资源占用极少
 - └─ 操作效率极高
 - └─ 易于脚本化
 - └─ 远程管理方便

结论：管理员选择 CLI 是因为效率！

效率对比演示

任务：找出 /var/log 目录下所有大于100M的文件

图形界面方式：

1. 打开文件管理器
 2. 导航到 /var/log
 3. 切换到列表视图
 4. 点击"大小"列排序
 5. 逐个查看文件大小
 6. 找出大于100M的文件
- 耗时：很快

命令行方式：

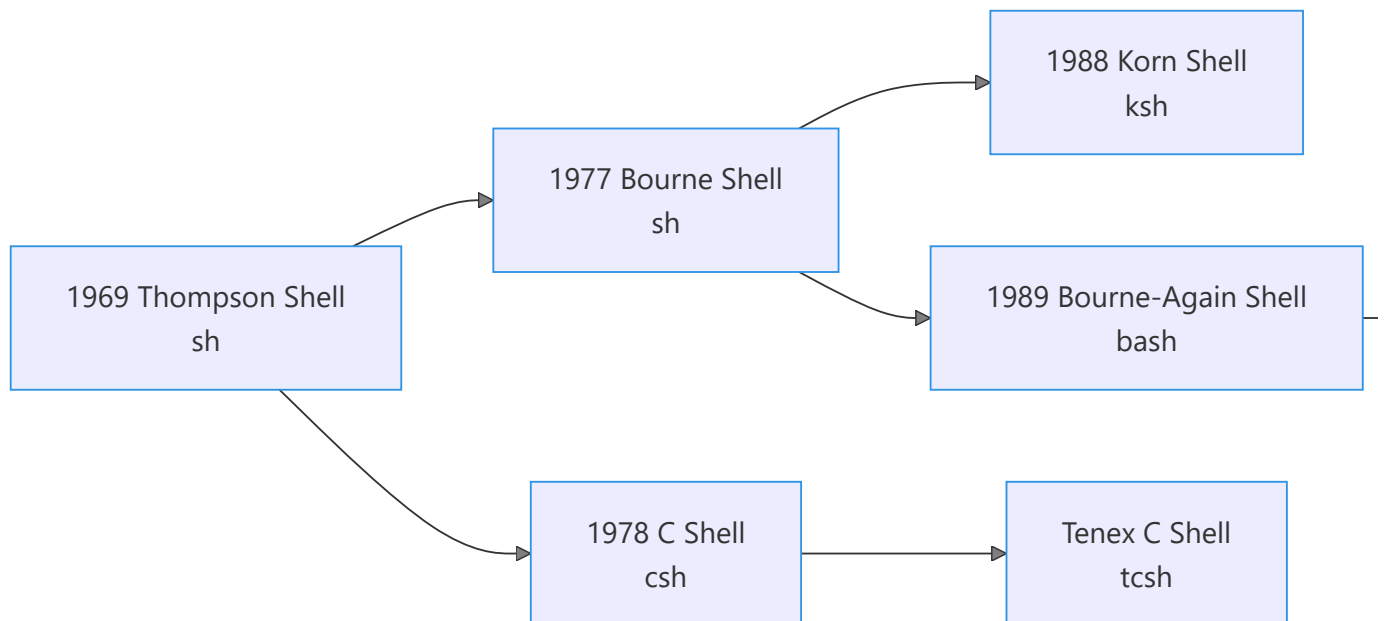
```
find /var/log -size +100M
```

耗时：约2秒

效率提升：约90倍！

4.2 Bash Shell 详解

Shell 的演变



Bash 的特点

Bash (Bourne-Again Shell)

- ├ 兼容 sh 的语法
- ├ 支持命令历史
- ├ 支持命令补全
- ├ 支持数组
- ├ 支持函数
- ├ 支持作业控制
- └ 内置编程语言

为什么叫 "Bourne-Again"?

- ├ 原始 Shell: Bourne Shell (sh)
- ├ 作者: Stephen Bourne
- ├ Bash 作者: Brian Fox
- └ 名字双关: Bourne Again = Born Again (重生)

Shell 提示符详解

```
# 提示符格式
[student@workstation ~]$
  ↓      ↓      ↓      ↓
用户名  主机名  当前目录 提示符

# 目录类型符号
~      # 家目录 (/home/student)
$      # 普通用户提示符
#      # root 用户提示符

# 自定义提示符 (PS1变量)
PS1='[\u@\h \W]\$'
# \u = 用户名
# \h = 主机名
# \W = 当前目录名
# \$ = 提示符

# 自定义提示符示例
PS1='[\u@\h \W]\$ '
# 输出: [student@workstation ~]$
```

练习名称	对应章节	说明
总练习：ACCESSING THE COMMAND LINE	第02章 57P	访问命令行、Shell基础操作、启动终端

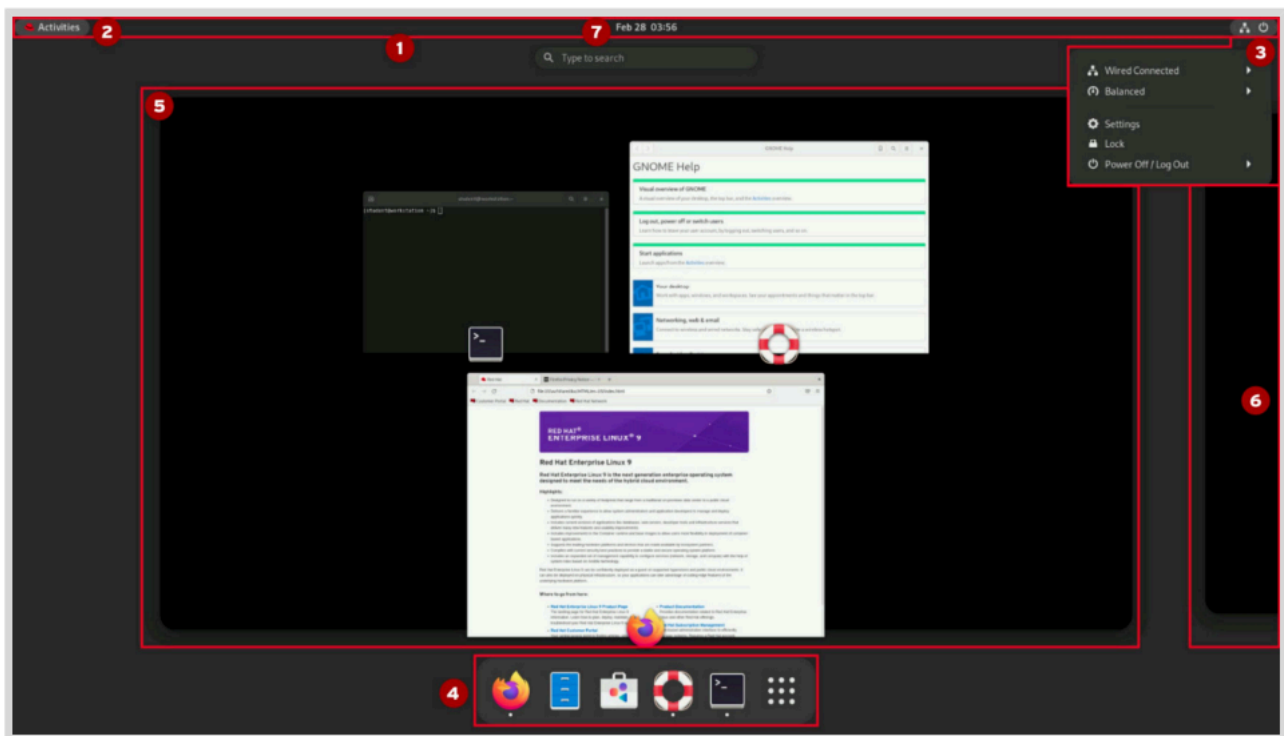
4.3 GNOME 桌面环境

什么是 GNOME？

```
GNOME = GNU Network Object Model Environment

RHEL9 默认桌面环境：
├─ GNOME 40 (最新版本)
├─ 显示协议：Wayland (默认) 或 X11
├─ 主题：Standard (标准) 或 Classic (经典)
└─ 特点：现代化、简洁、适合服务器管理
```

GNOME Shell 组成部分



组件说明：

1. Top Bar: 顶栏，系统信息和状态
2. Activities Overview: 活动概览（按Activities或Super键）
3. Dash: 常用应用和运行中的应用
4. Windows Overview: 所有窗口的缩略图
5. Workspace Selector: 工作区切换
6. Message Tray: 消息通知托盘

启动终端的多种方式

方式1: 通过 **Activities** 菜单

步骤:

1. 点击左上角 "Activities" 按钮
2. 在左侧 Dash 中点击 "Terminal" 图标

或

3. 在搜索框中输入 "terminal", 然后按 Enter

SHELL

方式2: 使用快捷键

打开运行命令窗口

Alt + F2

输入:

gnome-terminal

按 Enter

SHELL

方式3：右键菜单

```
# 在桌面空白处右键
# 选择 "Open in Terminal"
```

SHELL

终端窗口显示信息

```
┌ student@workstation: ~ ────────────┐
│ [student@workstation ~]$            │
│                                     │
│   ↑       ↑       ↑                 │
│ 用户名   主机名   当前目录           │
│                                     │
└──────────────────────────────────┘
```

终端标题栏显示：

- 当前用户名：student
- 主机名：workstation
- 当前工作目录：~（家目录）

GNOME 基本操作

锁定屏幕

```
# 快捷键
Super + L
# Super键 = Windows键或Command键

# 或通过系统菜单：
1. 点击右上角系统菜单
2. 点击 "Lock"
```

SHELL

注销登录

```
# 通过系统菜单：
1. 点击右上角系统菜单
2. 点击 "Log Out"
```

SHELL

关机或重启

方式1: 系统菜单

点击右上角系统菜单 → 电源按钮 → 选择操作

方式2: 快捷键

Ctrl + Alt + Del

→ 弹出对话框 → 选择 Power Off 或 Restart

方式3: 命令行 (后续课程详解)

poweroff # 关机

reboot # 重启

切换主题 (Standard ↔ Classic)

登录时操作:

1. 在密码输入框旁边点击  图标
2. 选择 "Standard" 或 "Classic"
3. 输入密码登录

区别:

- └─ Standard: 现代界面, 适合触控
- └─ Classic: 传统界面, 类似Windows

GNOME 快捷速查表

快捷键	功能
Super / Activities	打开活动概览
Super + L	锁定屏幕
Alt + F2	运行命令对话框
Alt + Tab	切换应用窗口
Super + Page Up/Down	切换工作区
Ctrl + Alt + 上/下	在工作区间移动窗口
Ctrl + Alt + Del	系统菜单 (关机/重启)
Ctrl + Q	关闭当前窗口

实践练习

练习1: 启动终端

尝试用三种不同方式打开终端:

方式1: 点击 Activities → 搜索 terminal

方式2: 按 Alt+F2 → 输入 gnome-terminal

方式3: 右键桌面 → Open in Terminal

练习2: 验证终端信息

在终端中查看当前用户和主机

echo "当前用户: \$(whoami)"

echo "当前主机: \$(hostname)"

echo "当前目录: \$(pwd)"

4.4 命令结构深度解析

命令的完整结构

命令的基本格式:

```
command [options] [arguments]
  ↓      ↓      ↓
  命令名 选项   参数
```

示例:

```
ls -la /home
  ↓  ↓  ↓
  命令 选项 参数
```

完整的命令结构 (后续课程会讲到) :

```
command [options] [arguments] [redirection] [background]
  ↓      ↓      ↓      ↓      ↓
  命令名 选项   参数   重定向  后台运行
```

说明:

- 重定向 (>): 将命令输出保存到文件
- 后台运行 (&): 命令在后台执行

这些高级功能将在后续课程中详细讲解

选项的两种形式

```
# 短选项 (单破折号 + 单字母)
ls -l -a
ls -la    # 可以合并

# 长选项 (双破折号 + 单词)
ls --all
ls --size --human-readable
ls --size --human-readable    # 不能合并

# 带参数的选项
ls -l /home
#   ↑   ↑
# 选项 参数
```

SHELL

常见错误分析

```
# 错误1: 命令拼写错误
ls /hme
# 错误: ls: cannot access '/hme': No such file or directory
# 原因: /hme 应该是 /home
# 解决: 使用Tab补全避免拼写错误

# 错误2: 选项顺序错误
ls -la /home
# ✓ 正确

ls /home -la
# ✓ 也正确 (大多数情况下)

# 错误3: 大小写错误
ls /Home
# 错误: ls: cannot access '/Home': No such file or directory
# 原因: Linux 区分大小写, 应该是 /home

# 错误4: 路径中的空格
ls /home/student/my documents
# 错误: ls: cannot access '/home/student/my': No such file or directory
#       ls: cannot access 'documents': No such file or directory
# 解决1: 使用引号
ls "/home/student/my documents"
# 解决2: 使用转义
ls /home/student/my\ documents
```

4.5 基本命令详解

查看系统信息命令

系统信息

```
# 查看完整的系统信息
uname -a
```

输出示例:

```
Linux workstation.lab.example.com 5.14.0-284.el9.x86_64 #1 SMP PREEMPT
  ↑           ↑           ↑           ↑
  内核名     主机名           内核版本     架构
```

系统详细信息

```
# 查看详细的系统信息
hostnamectl
```

输出示例:

```
Static hostname: workstation.lab.example.com
Icon name: computer-server
Chassis: server
Machine ID: ...
Boot ID: ...
Operating System: Red Hat Enterprise Linux 9.2 (Plow)
Kernel: Linux 5.14.0-284.el9.x86_64
Architecture: x86-64
```

发行版信息

```
# 查看发行版版本
cat /etc/redhat-release
```

SHELL

输出: Red Hat Enterprise Linux release 9.2 (Plow)

```
# 查看更详细的发行版信息
cat /etc/os-release
```

SHELL

内存信息

```
# 查看内存使用情况 (人类可读格式)
free -h
```

SHELL

参数说明: `-h` = human-readable, 以人类易读的格式显示 (如 GiB、MiB)

输出示例:

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	1.2Gi	1.5Gi	12Mi	1.1Gi	2.3Gi
Swap:	2.0Gi	0B	2.0Gi			

输出说明:

- `total` - 总内存
- `used` - 已使用内存
- `free` - 空闲内存
- `available` - 可用内存

CPU 信息

```
# 查看详细的 CPU 信息
lscpu
```

SHELL

磁盘信息

```
# 查看磁盘使用情况
df -h
```

SHELL

当前用户信息

```
# 查看当前用户名
whoami
```

SHELL

输出: **student**

```
# 查看当前用户的 UID、GID 等信息
id
```

SHELL

输出: **uid=1000(student) gid=1000(student) groups=1000(student)**

```
# 查看登录用户及其活动
w
```

SHELL

用户密码管理

passwd - 更改密码

```
# 更改当前用户的密码
passwd
```

SHELL

交互式流程示例:

```
[student@workstation ~]$ passwd
Changing password for user student.
Current password:      # 输入当前密码
New password:          # 输入新密码
Retype new password:   # 再次输入新密码确认
passwd: all authentication tokens updated successfully.
```

SHELL

输出说明:

- 需要先输入当前密码验证身份
- 新密码需要输入两次进行确认
- 密码输入时屏幕上不会显示任何字符（安全机制）

root 用户更改其他用户密码

```
# root 用户可以更改任何用户的密码，无需知道原密码
sudo passwd student
# 或直接使用 root
passwd student
```

SHELL

交互式流程：

```
[root@workstation ~]# passwd student
New password:          # 直接输入新密码（无需输入旧密码）
Retype new password:    # 再次确认
passwd: all authentication tokens updated successfully.
```

SHELL

passwd 命令选项

```
# -l 选项：锁定用户账户 (lock)
sudo passwd -l student
# 锁定后用户无法登录

# -u 选项：解锁用户账户 (unlock)
sudo passwd -u student

# -d 选项：删除用户密码 (delete)
# 删除后用户可以无密码登录（不安全，仅用于测试）
sudo passwd -d student

# -e 选项：强制用户下次登录时更改密码 (expire)
sudo passwd -e student
```

SHELL

日期时间命令

当前日期时间

```
# 查看当前日期和时间
date
```

SHELL

输出示例： `Mon Jan 15 14:30:45 CST 2024`

格式化输出

```
# 格式化输出日期 (YYYY-MM-DD)
date +%Y-%m-%d
```

输出： 2024-01-15

```
# 格式化输出时间 (HH:MM:SS)
date +%H:%M:%S
```

输出： 14:30:45

```
# 中文格式输出
date +"%Y年%m月%d日 %H:%M"
```

输出： 2024年01月15日 14:30

格式化选项参考

格式	说明	示例
%Y	4位年份	2024
%m	2位月份	01
%d	2位日期	15
%H	24小时制小时	14
%M	分钟	30
%S	秒	45
%u	星期几 (1-7, 1是周一)	1
%j	一年中的第几天	015

设定系统时间（需要 root 权限）

```
# 设置系统时间
# 注意：考试环境中通常不需要修改
date -s "2024-01-15 14:30:00"
```

日历命令

显示当月日历

```
cal
```

SHELL

显示全年日历

```
cal 2024
```

SHELL

显示指定月份的日历

```
cal 2 2024
```

SHELL

计算日期

计算下周一的日期

```
date -d "next monday"
```

SHELL

计算 2 天前的日期

```
date -d "2 days ago"
```

SHELL

计算 3 天后的日期

```
date -d "+3 days"
```

SHELL

文本处理命令

cat - 连接并显示文件

显示整个文件内容

```
cat /etc/passwd
```

SHELL

显示文件内容并添加行号

```
cat -n /etc/passwd
```

SHELL

```
# 连接并显示多个文件（按顺序显示）
cat /etc/hostname /etc/os-release
```

SHELL

说明：cat 会按顺序将多个文件的内容连接后显示

less - 分页查看文件

```
# 分页查看文件内容
less /etc/passwd
```

SHELL

在 less 中的操作：

- 空格 - 下一页
 - b - 上一页
 - /string - 搜索字符串
 - q - 退出
-

head - 查看文件开头

```
# 查看文件前 10 行（默认）
head /etc/passwd
```

SHELL

参数说明：默认显示前 10 行

```
# 查看文件前 5 行
head -n 5 /etc/passwd
```

SHELL

参数说明：-n 指定显示的行数，这里显示前 5 行

```
# 查看多个文件的前 3 行
head -n 3 /etc/passwd /etc/group
```

SHELL

参数说明：可以同时指定多个文件

tail - 查看文件结尾

```
# 查看文件后 10 行（默认）
tail /etc/passwd
```

SHELL

参数说明：默认显示后 10 行

SHELL

```
# 查看文件后 5 行
tail -n 5 /etc/passwd
```

参数说明：-n 指定显示的行数，这里显示后 5 行

SHELL

```
# 实时追踪日志文件变化
tail -f /var/log/messages
```

按 **Ctrl+C** 退出

参数说明：-f = follow，持续监控文件变化（常用于查看日志）

WC - 统计文件信息

SHELL

```
# 统计文件的行数、字数、字节数
wc /etc/passwd
```

输出示例：

```
65    65 3079 /etc/passwd
  ↓    ↓    ↓
行数 字数 字节数
```

SHELL

```
# 只统计行数
wc -l /etc/passwd
```

参数说明：-l = lines，只显示行数

SHELL

```
# 只统计字数
wc -w /etc/passwd
```

参数说明：-w = words，只显示字数

SHELL

```
# 只统计字节数
wc -c /etc/passwd
```

参数说明: `-c` = bytes, 只显示字节数

SHELL

```
# 统计多个文件
wc -l /etc/passwd /etc/group
```

参数说明: 可以同时统计多个文件

file - 识别文件类型

SHELL

```
# 识别文件类型
file /etc/passwd
```

输出: `/etc/passwd: ASCII text`

SHELL

```
# 识别可执行文件
file /bin/ls
```

输出: `/bin/ls: ELF 64-bit LSB pie executable, x86-64, ...`

SHELL

```
# 识别目录
file /home
```

输出: `/home: directory`

SHELL

```
# 识别设备文件
file /dev/sda
```

输出: `/dev/sda: block special`

练习名称	对应章节	说明
总练习: ACCESSING THE COMMAND LINE	第02章	系统信息查看、基础命令操作

4.6 文件管理基础

本节目标

- 了解 Linux 文件系统结构
- 掌握基本的文件和目录操作命令
- 学会使用 pwd、cd、ls、mkdir 等基础命令

注意： 本节是第3章"使用命令行管理文件"的基础部分，完整内容将在Day 2继续学习。

Linux 文件系统结构

Linux 文件系统是树形结构：

```
/ (根目录)
├─ bin      # 常用命令
├─ boot     # 启动文件
├─ dev      # 设备文件
├─ etc      # 配置文件
├─ home     # 普通用户家目录
│   └─ student
│   └─ user1
├─ root     # root 用户家目录
├─ tmp      # 临时文件
├─ usr      # 软件和文档
└─ var      # 可变数据（日志等）
```

pwd - 显示当前工作目录

```
# pwd = Print Working Directory
pwd
```

SHELL

输出示例：

```
[student@workstation ~]$ pwd
/home/student
```

SHELL

说明：

- 显示当前所在的完整路径
 - ~ 符号代表用户家目录（/home/用户名）
 - root 用户的家目录是 /root
-

cd - 切换目录

```
# cd = Change Directory
# 切换到指定目录
cd /etc

# 切换到用户家目录
cd
# 或
cd ~

# 切换到上一级目录
cd ..

# 切换到上一次所在目录
cd -

# 切换到指定用户的家目录
cd ~student
```

路径说明：

绝对路径 vs 相对路径

绝对路径：

- └ 从根目录 / 开始
- └ 例如：/home/student/Documents
- └ 任何位置都可以使用

相对路径：

- └ 从当前目录开始
- └ 例如：Documents/report.txt
- └ . 表示当前目录
- └ .. 表示上级目录
- └ 依赖于当前位置

示例：

当前在 /home/student

cd Documents # 相对路径，进入 /home/student/Documents

cd /home/student/Documents # 绝对路径，从任何位置都有效

ls - 列出目录内容

```
[student@workstation ~]$ ls -lh
total 4.0K
drwxrwxr-x. 2 student student 4.0K Jan 15 10:30 Documents
drwxrwxr-x. 2 student student 4.0K Jan 15 10:30 Downloads
-rw-rw-r--. 1 student student 123 Jan 15 10:30 file.txt
lrwxrwxrwx. 1 student student 10 Jan 15 10:30 link -> file.txt
```

mkdir - 创建目录

```
# mkdir = Make Directory
# 创建单个目录
mkdir project

# 创建多个目录
mkdir dir1 dir2 dir3

# -p 选项: 创建多级目录
mkdir -p /tmp/project/docs/notes

# -v 选项: 显示创建过程 (verbose)
mkdir -v newdir
# 输出: mkdir: created directory 'newdir'
```

使用场景:

```
# 场景1: 创建项目目录结构
mkdir -p project/{src,docs,tests}
# 创建: project/src, project/docs, project/tests

# 场景2: 创建带日期的目录
mkdir -p logs/$(date +%Y%m%d)
# 创建: logs/20240115 (假设今天是2024年1月15日)
```

touch - 创建空文件

```
# 创建一个空文件
touch file.txt

# 创建多个文件
touch file1.txt file2.txt file3.txt

# 创建带大括号扩展的文件
touch file{1..10}.txt
# 创建: file1.txt, file2.txt, ..., file10.txt
```

rm - 删除文件和目录

⚠️ 警告: rm 命令删除的文件无法恢复! 请谨慎使用!

```
# 删除文件
rm file.txt

# 删除多个文件
rm file1.txt file2.txt

# -i 选项: 删除前确认 (interactive)
rm -i file.txt
# 输出: rm: remove regular file 'file.txt'? 输入 y 确认

# -r 选项: 递归删除目录
rm -r directory

# -f 选项: 强制删除, 不确认 (危险!)
rm -f file.txt

# 组合使用: 递归强制删除目录
rm -rf directory
# ⚠️ 极其危险, 请确保路径正确!
```

安全建议:

```
# 推荐做法: 使用 -i 选项
rm -i important_file.txt

# 绝对路径确认, 防止误删
rm -i /path/to/file.txt

# 定期备份重要数据 (后续课程会讲)
```

文件管理实践练习

练习1: 目录导航

```
# 1. 查看当前位置
pwd

# 2. 切换到 /etc 目录
cd /etc

# 3. 查看 /etc 目录内容
ls

# 4. 返回家目录
cd

# 5. 确认已回到家目录
pwd
```

练习2: 创建目录结构

```
# 1. 在家目录创建项目目录
mkdir -p ~/project/{src,docs,tests}

# 2. 验证目录结构
ls -R ~/project

# 3. 切换到 src 目录
cd ~/project/src

# 4. 创建测试文件
touch test1.py test2.py

# 5. 返回项目根目录
cd ~/project
```

练习3：文件管理综合

```
# 1. 创建临时目录
mkdir -p ~/temp/work

# 2. 在目录中创建多个文件
cd ~/temp/work
touch {a,b,c}.txt

# 3. 查看创建的文件
ls -l

# 4. 删除单个文件（带确认）
rm -i a.txt

# 5. 删除整个 work 目录
cd ~/temp
rm -ri work
# 每个文件都会要求确认
```

常见错误与解决

```
# 错误1: 目录不存在
cd /nonexistent
# 错误: bash: cd: /nonexistent: No such file or directory

# 错误2: 权限不足
mkdir /root/testdir
# 错误: mkdir: cannot create directory '/root/testdir': Permission denied
# 解决: 需要使用 sudo 或切换到 root 用户

# 错误3: 删除目录时忘记 -r
rm directory
# 错误: rm: cannot remove 'directory': Is a directory
# 解决: 使用 rm -r directory

# 错误4: 路径中的空格
ls my documents
# 错误: ls: cannot access 'my': No such file or directory
# 解决1: 使用引号 ls "my documents"
# 解决2: 使用转义 ls my\ documents
# 解决3: 使用Tab补全
```

4.7 命令效率技巧

Tab 补全魔法

基础补全

```
# 输入命令的开头, 按 Tab 键自动补全
pas<TAB>
```

补全为: `passwd`

```
# 补全路径
cat /etc/pas<TAB>
```

补全为: `cat /etc/passwd`

显示所有匹配

```
# 如果有多个匹配, 按两次 Tab 显示所有选项
ls /etc/sysco<TAB><TAB>
```

显示: 所有以 `sysco` 开头的文件

补全选项

```
# 补全命令选项
ls --<TAB><TAB>
```

SHELL

显示: `ls` 命令的所有选项

补全路径

```
# 补全目录路径
cd /home/stu<TAB>
```

SHELL

补全为: `cd /home/student/`

实用技巧

技巧1: 快速检查路径是否存在

```
# 尝试补全一个不存在的路径
cat /path/to/nonexistent<TAB>
# 没有反应 = 路径不存在
```

SHELL

技巧2: 避免拼写错误

```
# 手动输入容易出错
# Tab补全确保正确
```

SHELL

技巧3: 提高效率

```
# 输入越少, 速度越快
# 使用 Tab 补全可以节省大量时间
```

SHELL

命令历史威力

查看历史

```
# 查看最近 1000 条命令 (默认)
history
```

SHELL

```
# 查看最近 20 条命令
history 20
```

SHELL

执行历史命令

```
# 重复上一条命令
!!
```

SHELL

```
# 重复最近一条以 cat 开头的命令
!cat
```

SHELL

```
# 执行第 100 条命令
!100
```

SHELL

搜索历史

```
# 按 Ctrl+R, 然后输入关键字搜索
# 继续按 Ctrl+R 查找更早的匹配
```

SHELL

使用箭头键

```
# 按上箭头: 查看上一条命令
# 按下箭头: 查看下一条命令
# 找到后按 Enter 执行
```

SHELL

Bash 基础快捷键

光标移动

快捷键	功能
Ctrl+A	移到行首
Ctrl+E	移到行尾
左右箭头	逐字符移动

编辑命令

快捷键	功能
Ctrl+U	删除到行首
Ctrl+K	删除到行尾
Ctrl+W	删除一个单词
Ctrl+Y	粘贴删除的内容

进程控制

快捷键	功能
Ctrl+C	终止当前命令
Ctrl+L	清屏（等于输入 <code>clear</code> ）
Ctrl+D	退出 Shell（等于输入 <code>exit</code> ）

历史控制

快捷键	功能
Ctrl+R	搜索历史
上箭头	上一条命令
下箭头	下一条命令

今日总结

上午知识点回顾

- RHCSA 认证体系和考试要求
 - 课程环境架构和网络配置
 - 虚拟机管理命令 (rht-vmctl)
 - /etc/hosts 配置
 - SSH 连接配置和优化
-

下午知识点回顾

- Linux 的历史和应用领域
- 开源软件的概念和许可证
- Linux 发行版和红帽生态系统
- Bash Shell 基础和命令结构
- 常用系统管理命令
- 命令行效率技巧

实践任务清单

SHELL

必做任务

- 配置所有虚拟机的 /etc/hosts
- 熟练使用 SSH 连接各虚拟机
- 完成系统信息收集练习
- 练习 Tab 补全和命令历史

选做任务

- 优化 SSH 配置文件
- 自定义 Bash 提示符
- 创建个人命令速查表
- 配置命令别名提高效率

RHCSA 考试要点

今天的知识点与考试的关系：

Day 01 内容	考试相关性
└─ RHCSA 认证体系	直接相关（了解考试形式）
└─ 环境配置	间接相关（考试环境类似）
└─ SSH 连接	★★★ 非常重要（考试必用）
└─ 命令行基础	★★★ 非常重要（所有题目基础）
└─ 系统信息命令	★★ 重要（题目中经常需要）
└─ Tab 补全和快捷键	★★ 重要（提高效率）
└─ 命令历史	★★ 重要（提高效率）

考试提醒：

1. SSH 连接是考试的基础
 - 考试时需要通过 SSH 连接到多台服务器
 - 必须熟练使用 `ssh user@hostname` 命令
 - 主机名拼写错误会导致严重失分！
2. 命令行效率决定成绩

- 考试时间3小时，约15-20题
- 合理分配每题时间
- Tab 补全和快捷键能节省大量时间

今日必会命令：

```
# === 考试生存命令 ===  
ssh user@host      # SSH连接（最重要）  
hostname           # 确认当前主机  
exit               # 退出SSH  
Tab                # 命令补全（防拼写错误）  
Ctrl+C            # 终止当前命令  
上下箭头          # 查看命令历史
```

SHELL